

Hibernate Entity Relationships Reference

20 table pairs mapped to JPA/Hibernate relationship annotations (OneToOne, OneToMany/ManyToOne, ManyToMany), with reasoning and code for both sides.

#	Table A	Table B	Relationship
1	Student	Passport	OneToOne
2	Employee	Department	ManyToOne / OneToMany
3	Customer	Order	OneToMany / ManyToOne
4	Book	Author	ManyToMany
5	Teacher	Subject	ManyToMany
6	User	Role	ManyToMany
7	Course	Student	ManyToMany
8	Library	Book	OneToMany / ManyToOne
9	Doctor	Patient	ManyToMany
10	Product	Category	ManyToOne / OneToMany
11	Project	Employee	ManyToMany
12	Invoice	Payment	OneToMany / ManyToOne
13	Flight	Ticket	OneToMany / ManyToOne
14	Movie	Actor	ManyToMany
15	Parent	Child	OneToMany / ManyToOne
16	School	Classroom	OneToMany / ManyToOne
17	Driver	Car	OneToOne
18	Hotel	Room	OneToMany / ManyToOne
19	Account	Transaction	OneToMany / ManyToOne
20	Country	City	OneToMany / ManyToOne

1. Student ↔ Passport

OneToOne

A student holds exactly one passport, and a passport belongs to exactly one student.

```
@Entity
public class Student {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToOne(mappedBy = "student", cascade = CascadeType.ALL)
    private Passport passport;
}

@Entity
public class Passport {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String passportNumber;

    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "student_id", unique = true)
    private Student student;
}
```

2. Employee ↔ Department

ManyToOne / OneToMany

Many employees work in one department; a department contains many employees. Employee is the owning side (holds the FK).

```
@Entity
public class Employee {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")
    private Department department;
}

@Entity
public class Department {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL)
    private List<Employee> employees = new ArrayList<>();
}
```

3. Customer ↔ Order

OneToMany / ManyToOne

One customer can place many orders; each order belongs to exactly one customer. Order is the owning side.

```
@Entity
public class Customer {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(mappedBy = "customer", cascade = CascadeType.ALL)
    private List<Order> orders = new ArrayList<>();
}
```

```

@Entity
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private LocalDate orderDate;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "customer_id")
    private Customer customer;
}

```

4. Book ↔ Author

ManyToMany

A book can be written by several authors, and an author can write several books.

```

@Entity
public class Book {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;

    @ManyToMany
    @JoinTable(
        name = "book_author",
        joinColumns = @JoinColumn(name = "book_id"),
        inverseJoinColumns = @JoinColumn(name = "author_id")
    )
    private List<Author> authors = new ArrayList<>();
}

```

```

@Entity
public class Author {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @ManyToMany(mappedBy = "authors")
    private List<Book> books = new ArrayList<>();
}

```

5. Teacher ↔ Subject

ManyToMany

A teacher can teach several subjects, and a subject can be taught by several teachers.

```

@Entity
public class Teacher {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @ManyToMany
    @JoinTable(
        name = "teacher_subject",
        joinColumns = @JoinColumn(name = "teacher_id"),
        inverseJoinColumns = @JoinColumn(name = "subject_id")
    )
    private List<Subject> subjects = new ArrayList<>();
}

```

```

@Entity
public class Subject {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    @ManyToMany(mappedBy = "subjects")
}

```

```

    private List<Teacher> teachers = new ArrayList<>();
}

```

6. User ↔ Role

ManyToMany

A user can have several roles (e.g. ADMIN, USER), and a role can be assigned to several users.

```

@Entity
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String username;

    @ManyToMany
    @JoinTable(
        name = "user_role",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id")
    )
    private List<Role> roles = new ArrayList<>();
}

@Entity
public class Role {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String roleName;

    @ManyToMany(mappedBy = "roles")
    private List<User> users = new ArrayList<>();
}

```

7. Course ↔ Student

ManyToMany

A course has many enrolled students, and a student can enroll in many courses.

```

@Entity
public class Course {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;

    @ManyToMany
    @JoinTable(
        name = "course_student",
        joinColumns = @JoinColumn(name = "course_id"),
        inverseJoinColumns = @JoinColumn(name = "student_id")
    )
    private List<Student> students = new ArrayList<>();
}

@Entity
public class Student {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @ManyToMany(mappedBy = "students")
    private List<Course> courses = new ArrayList<>();
}

```

8. Library ↔ Book

OneToMany / ManyToOne

A library holds many books; each book belongs to exactly one library. Book is the owning side.

```

@Entity
public class Library {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(mappedBy = "library", cascade = CascadeType.ALL)
    private List<Book> books = new ArrayList<>();
}

@Entity
public class Book {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "library_id")
    private Library library;
}

```

9. Doctor ↔ Patient

ManyToMany

A doctor treats many patients over time, and a patient may be treated by several doctors (specialists, referrals, etc.).

```

@Entity
public class Doctor {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @ManyToMany
    @JoinTable(
        name = "doctor_patient",
        joinColumns = @JoinColumn(name = "doctor_id"),
        inverseJoinColumns = @JoinColumn(name = "patient_id")
    )
    private List<Patient> patients = new ArrayList<>();
}

@Entity
public class Patient {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @ManyToMany(mappedBy = "patients")
    private List<Doctor> doctors = new ArrayList<>();
}

```

10. Product ↔ Category

ManyToOne / OneToMany

Each product belongs to exactly one category; a category groups many products. Product is the owning side.

```

@Entity
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "category_id")
    private Category category;
}

@Entity

```

```

public class Category {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(mappedBy = "category", cascade = CascadeType.ALL)
    private List<Product> products = new ArrayList<>();
}

```

11. Project ↔ Employee

ManyToMany

A project has several employees assigned to it, and an employee can work on several projects.

```

@Entity
public class Project {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @ManyToMany
    @JoinTable(
        name = "project_employee",
        joinColumns = @JoinColumn(name = "project_id"),
        inverseJoinColumns = @JoinColumn(name = "employee_id")
    )
    private List<Employee> employees = new ArrayList<>();
}

@Entity
public class Employee {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @ManyToMany(mappedBy = "employees")
    private List<Project> projects = new ArrayList<>();
}

```

12. Invoice ↔ Payment

OneToMany / ManyToOne

An invoice can be settled with several payments (partial/installment payments); each payment applies to exactly one invoice.

```

@Entity
public class Invoice {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private BigDecimal totalAmount;

    @OneToMany(mappedBy = "invoice", cascade = CascadeType.ALL)
    private List<Payment> payments = new ArrayList<>();
}

@Entity
public class Payment {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private BigDecimal amount;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "invoice_id")
    private Invoice invoice;
}

```

13. Flight ↔ Ticket

OneToMany / ManyToOne

A flight has many tickets sold for it; each ticket is issued for exactly one flight.

```
@Entity
public class Flight {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String flightNumber;

    @OneToMany(mappedBy = "flight", cascade = CascadeType.ALL)
    private List<Ticket> tickets = new ArrayList<>();
}

@Entity
public class Ticket {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String seatNumber;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "flight_id")
    private Flight flight;
}
```

14. Movie ↔ Actor

ManyToMany

A movie has several actors in its cast, and an actor appears in several movies.

```
@Entity
public class Movie {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;

    @ManyToMany
    @JoinTable(
        name = "movie_actor",
        joinColumns = @JoinColumn(name = "movie_id"),
        inverseJoinColumns = @JoinColumn(name = "actor_id")
    )
    private List<Actor> actors = new ArrayList<>();
}

@Entity
public class Actor {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @ManyToMany(mappedBy = "actors")
    private List<Movie> movies = new ArrayList<>();
}
```

15. Parent ↔ Child

OneToMany / ManyToOne

A parent can have many children; each child record links back to one parent (model a second parent with a similar field, or use ManyToMany if both parents share the link).

```
@Entity
public class Parent {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
}
```

```

    @OneToMany(mappedBy = "parent", cascade = CascadeType.ALL)
    private List<Child> children = new ArrayList<>();
}

@Entity
public class Child {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "parent_id")
    private Parent parent;
}

```

16. School ↔ Classroom

OneToMany / ManyToOne

A school contains many classrooms; each classroom belongs to exactly one school.

```

@Entity
public class School {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(mappedBy = "school", cascade = CascadeType.ALL)
    private List<Classroom> classrooms = new ArrayList<>();
}

@Entity
public class Classroom {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String roomNumber;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "school_id")
    private School school;
}

```

17. Driver ↔ Car

OneToOne

In a simple ride/fleet model, a driver is currently assigned exactly one car, and that car is assigned to exactly one driver at a time.

```

@Entity
public class Driver {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToOne(mappedBy = "driver", cascade = CascadeType.ALL)
    private Car car;
}

@Entity
public class Car {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String licensePlate;

    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "driver_id", unique = true)
    private Driver driver;
}

```

```
}
```

18. Hotel ↔ Room

OneToMany / ManyToOne

A hotel has many rooms; each room belongs to exactly one hotel.

```
@Entity
public class Hotel {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(mappedBy = "hotel", cascade = CascadeType.ALL)
    private List<Room> rooms = new ArrayList<>();
}
```

```
@Entity
public class Room {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String roomNumber;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "hotel_id")
    private Hotel hotel;
}
```

19. Account ↔ Transaction

OneToMany / ManyToOne

An account has many transactions recorded against it; each transaction belongs to exactly one account.

```
@Entity
public class Account {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String accountNumber;

    @OneToMany(mappedBy = "account", cascade = CascadeType.ALL)
    private List<Transaction> transactions = new ArrayList<>();
}
```

```
@Entity
public class Transaction {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private BigDecimal amount;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "account_id")
    private Account account;
}
```

20. Country ↔ City

OneToMany / ManyToOne

A country has many cities; each city belongs to exactly one country.

```
@Entity
public class Country {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(mappedBy = "country", cascade = CascadeType.ALL)
    private List<City> cities = new ArrayList<>();
}
```

```
}

@Entity
public class City {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "country_id")
    private Country country;
}
```